

VOID-NAV

When the towers fall, every phone can still call for help.

A LoRa relay network that phones connect to over local Wi-Fi.
No cellular, no internet, no app. Messages are prioritised, kept jam-free, and confirmed back to the sender.

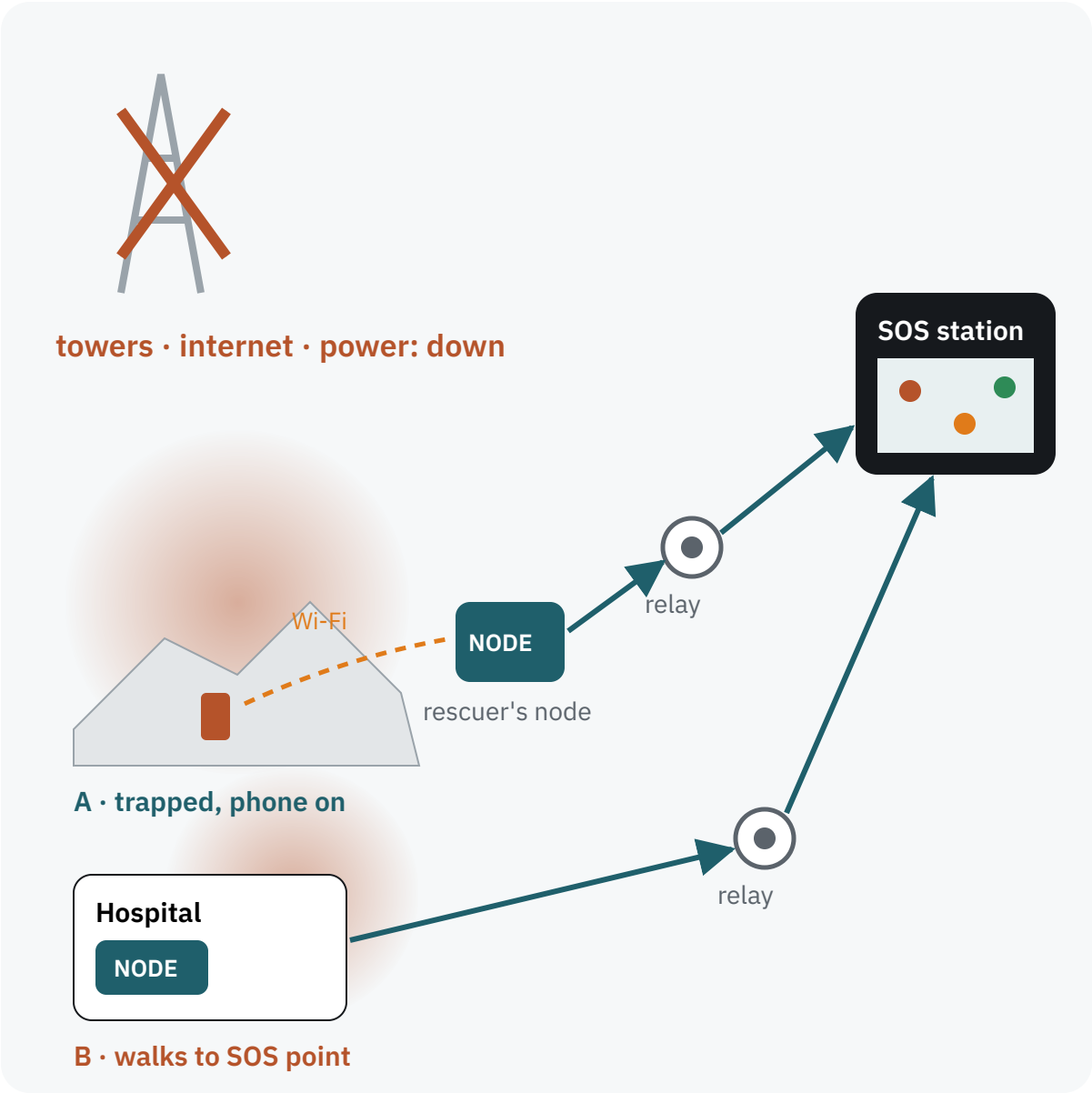
- SCENARIO A Trapped with a phone → **rescuers bring the network**
- SCENARIO B Can move → **walk to a public SOS point**

TEAM

Team VOID

TEAM ID

[to be filled]



After a disaster, everyone needs to talk. That's exactly when the network disappears.

PAIN 1

Trapped, with a phone that can't call

Under rubble or stranded by water. The phone works, but there's no network to reach anyone.

PAIN 2

Rescuers search blind

Nobody knows where people are, how many there are, or who is most critical.

PAIN 3

Emergency networks jam

If every relay rebroadcasts every message, a small radio channel chokes exactly when traffic peaks.

We need a way to send SOS messages with **zero infrastructure**, using the **phones people already have**, that tells the sender the message **arrived** and **stays fast** when hundreds of people use it at once.

Who is the survivor, and who brings the network?

SCENARIO A

Trapped, but has a phone

Under rubble, on a rooftop, cut off by flood. Can't move.

Rescuers bring the network to them:

- carry nodes into the area
- place relays where **quantum planning** says
- detect phones, open "SOS" Wi-Fi
- see everything on an offline dashboard

SCENARIO B

Can move, may have no phone

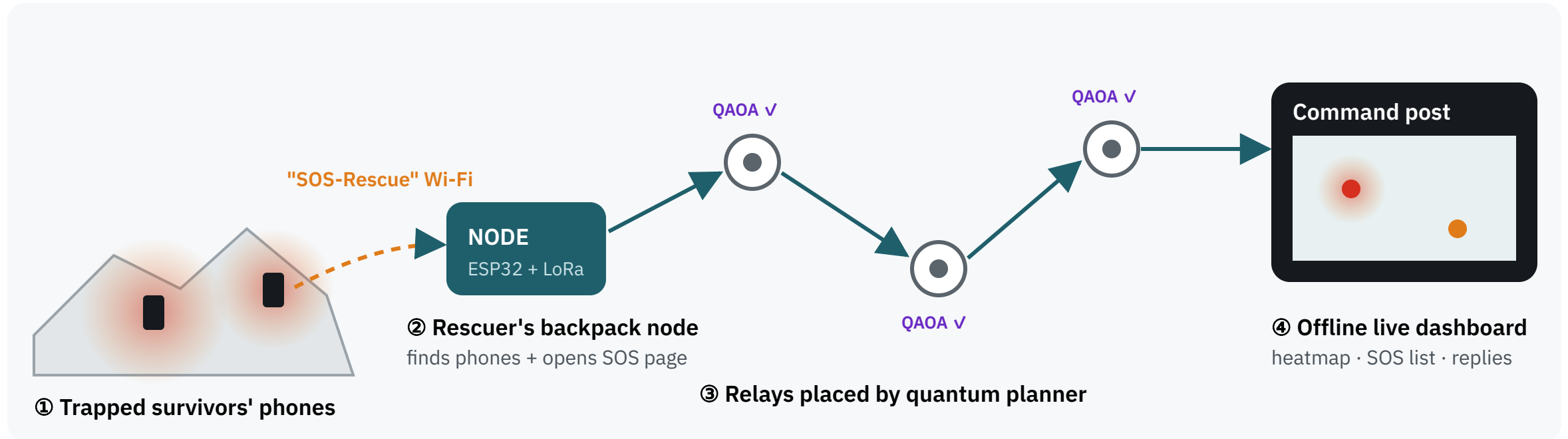
Mobile but cut off: a lost or dead phone, or no network.

They go to a public SOS point:

- nodes pre-placed at hospitals, panchayats, stations
- alert + map tell people where
- use own phone, or the **shared phone/tablet** kept there
- same relays, same dashboard

Both use the **same node** (ESP32 + LoRa), the **same relay network** and the **same dashboard**. Only who places the node changes.

Rescuers carry the network in



FIND

Phones keep sending Wi-Fi/Bluetooth signals. The node logs signal strength + GPS → a **heatmap of where phones are**.

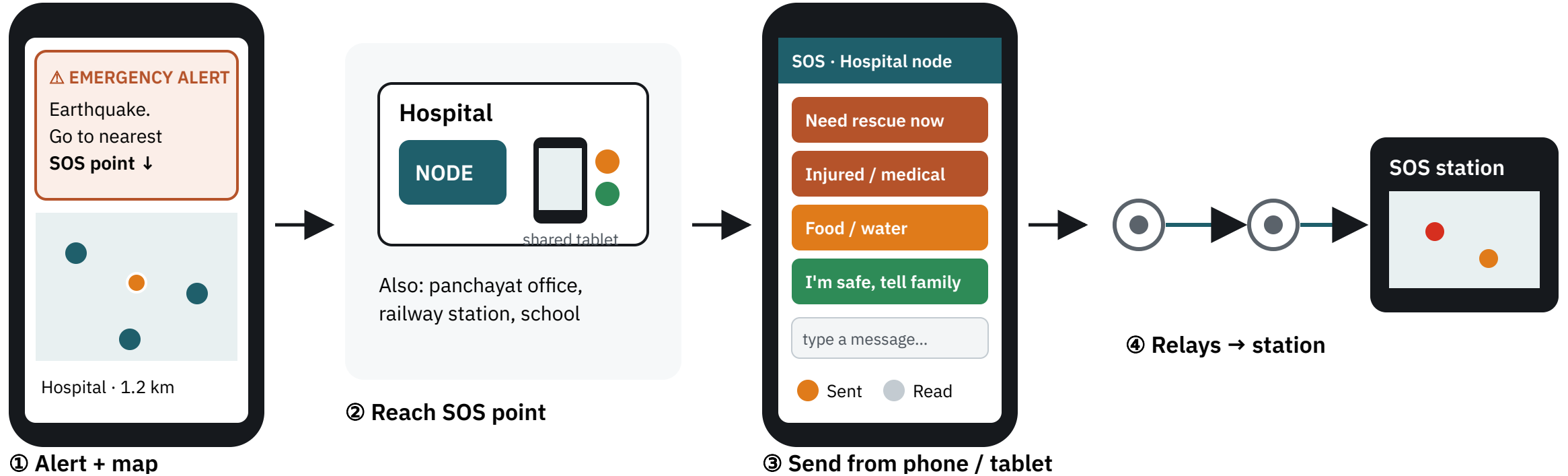
TALK

Survivor joins open Wi-Fi "SOS-Rescue"; a help page pops up by itself. No app, no internet.

REPLY

Rescuers reply from the dashboard: "*Team coming, stay where you are.*" The survivor sees it on the same page.

Public SOS points, pre-placed and announced

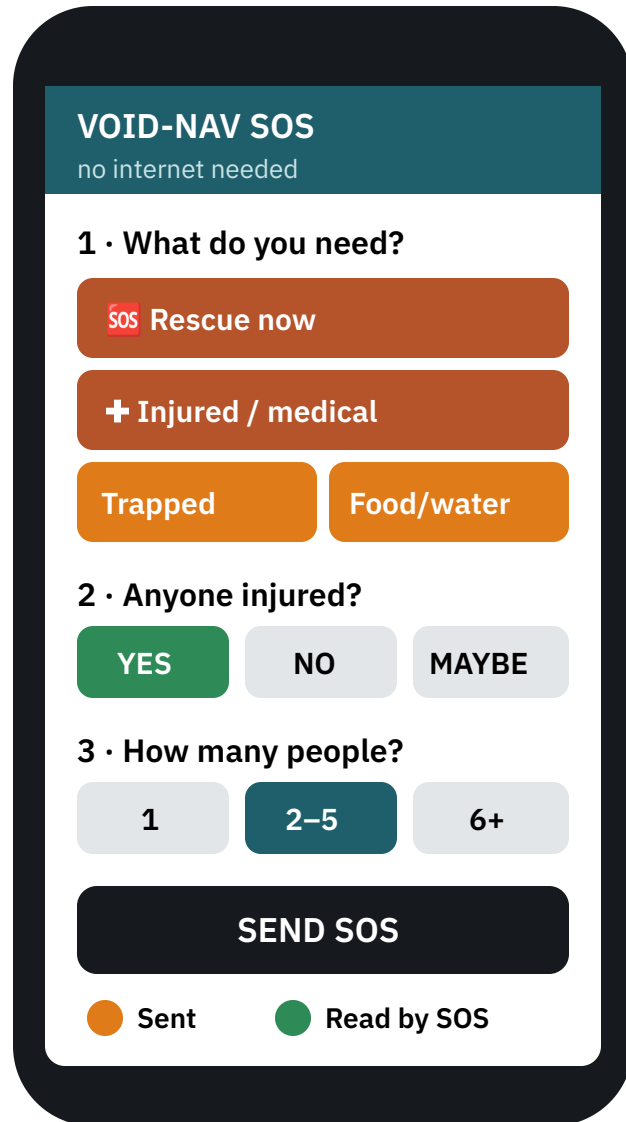


No phone? Use the shared tablet at the point, or ask staff there. Same page, same node.

Location: the node's coordinates are saved when it's installed, labelled "Hospital node 07".

Alert: attaching "nearest SOS points" to official alerts is a **proposed** integration.

No app. Connect to Wi-Fi, and the SOS page opens.



VOID-NAV SOS
no internet needed

1 • What do you need?

SOS Rescue now

+ Injured / medical

Trapped Food/water

2 • Anyone injured?

YES NO MAYBE

3 • How many people?

1 2-5 6+

SEND SOS

● Sent ● Read by SOS

QUICK BUTTONS FIRST

Most urgent options at the top. One tap + 2 questions. Answers fit in a **~21-byte** message.

YES / NO / MAYBE

Short questions anyone can answer under stress. Local languages supported.

CUSTOM TEXT

Optional short note (≤ 40 characters) to the SOS station **or to family at another node.**

LOCATION

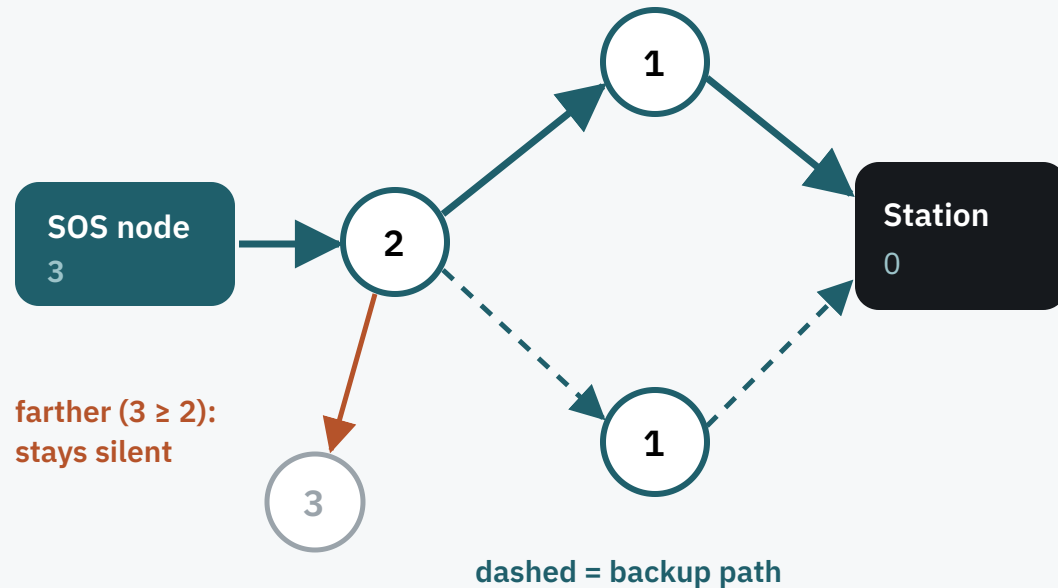
Node's saved coordinates or its GPS, clearly labelled. Phone GPS isn't relied on (browsers block it on plain local pages).

TWO STATUS LIGHTS • ON THE PAGE AND ON THE NODE

- **Orange: Sent.** The node transmitted it. The page then shows "Delivered ✓" when the station's automatic receipt comes back.
- **Green: Read by SOS.** An operator opened it and assigned help. This only happens after a person acts, never automatically.

Fast for urgent cases, never jammed

Number = hops to the SOS station



21-byte packet $\approx$ 185 ms on air (SF9, 125 kHz, calculated)

1 • FORGET REPEATS

Unique message ID; relays drop copies.

2 • HOP LIMIT + EXPIRY

Max 4 hops; old messages expire.

3 • CLOSER-ONLY

Forward only if nearer the station. No blind flooding.

4 • PRIORITY LANES

Rescue/medical first; waiting ones move up.

5 • LISTEN, THEN TALK

Check the channel, short random wait, cancel if already sent.

6 • CAPPED RETRIES

Retry with growing gaps, then stop.

≈ 0.3 s

per hop, urgent (est.)

≈ 1.6 s

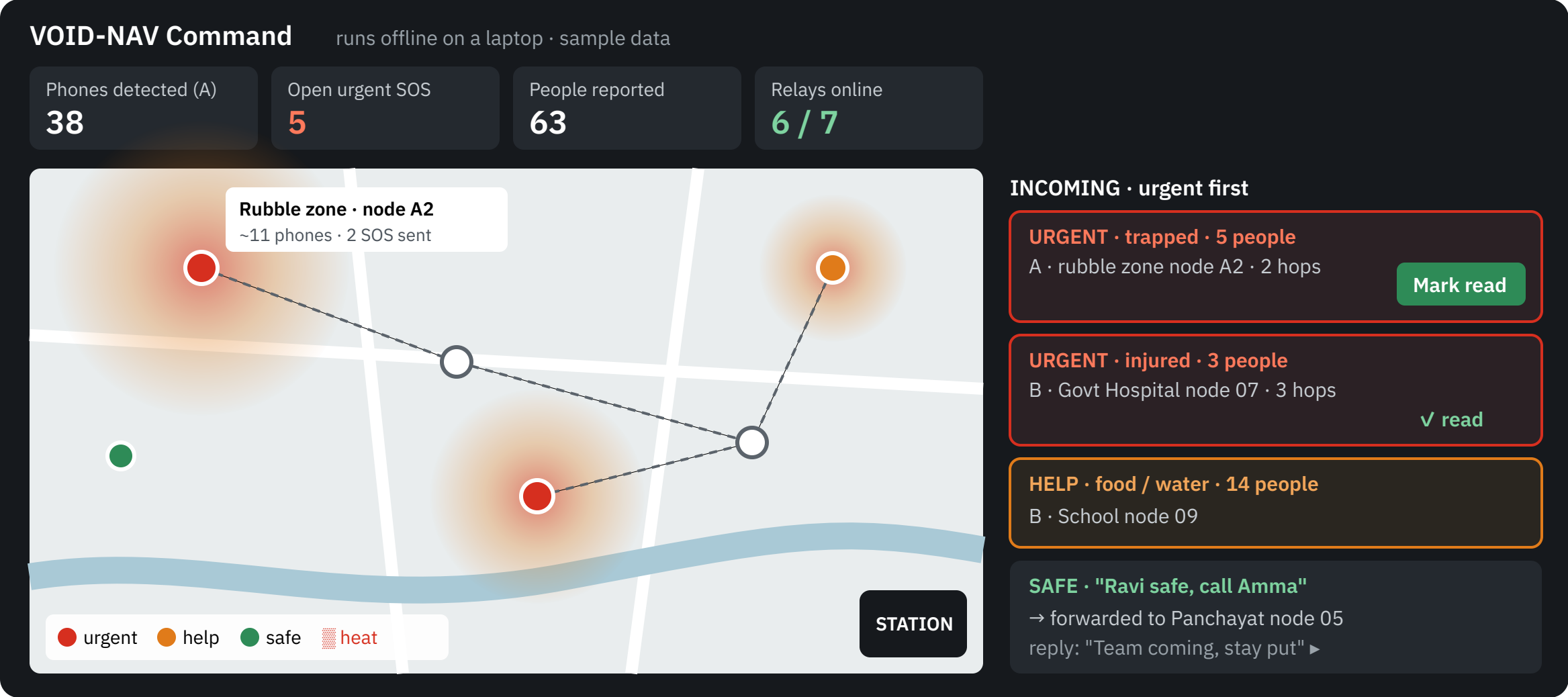
3 hops + receipt (est.)

1 channel

more relays $\neq$ more capacity

Estimates, to be measured on our prototype. Collisions are reduced, not eliminated.

Who needs help, where, and how urgent

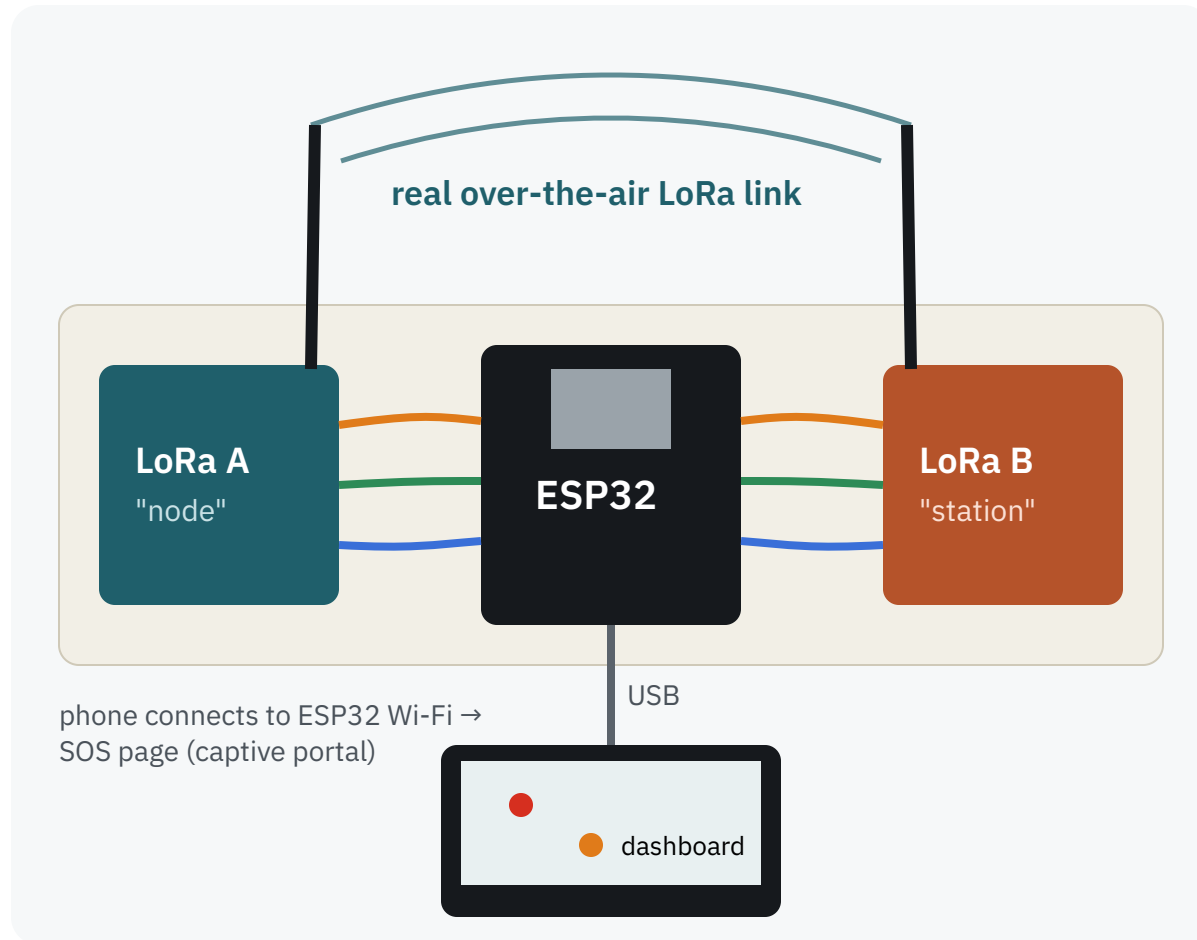


What's the same, what's different, what's real

	A · TRAPPED + PHONE	B · PUBLIC SOS POINT	STATUS
Who places nodes	rescue teams, carried in after the event	authorities, installed before disasters	buildable
How a message is typed	survivor's own phone → "SOS-Rescue" Wi-Fi page	own phone, or shared tablet at the point	buildable
Finding people	phone-signal scan → heatmap (counts, no IDs)	people come to the point	buildable*
Location shown	rescuer node GPS + signal strength (rough area)	saved coordinates of the fixed node	buildable
Relay placement	chosen with QAOA quantum planner	chosen in preparedness planning (QAOA can help)	Qiskit sim
How people find it	open Wi-Fi name "SOS-Rescue"	official alert + map of SOS points	alert: proposed
Shared by both	same ESP32 + LoRa node · same 6 traffic rules · same orange/green status · same offline dashboard		buildable

*Modern phones randomise Wi-Fi IDs, so the scan estimates how many phones and roughly where, not who. Phones that are off or in airplane mode can't be detected. "Buildable" = standard ESP32/LoRa capability; to be shown in our demo or next build, not yet field-tested.

1 ESP32 · 2 LoRa radios · 2 antennas · breadboard



DEMO SCRIPT · 3 MINUTES

1. Judge joins Wi-Fi "**SOS-Rescue**"; the SOS page opens
2. Taps "Rescue now"; LoRa A sends it over the air; **orange**
3. LoRa B receives it; a pin appears on the dashboard
4. We click "Mark read"; the reply comes back; **green**
5. Resend the same ID: dropped. Urgent jumps the queue.

Honest scope: one ESP32 runs both radios (shared SPI, separate CS pins), so this proves the message, receipt, status lights, priority and duplicate logic on a real radio link. **Real relay hops need 1 ESP32 + 1 LoRa per node (≥ 3 boards):** our next step.

Pins: SPI 18/19/23 shared · A: CS 5, RST 14, DIO0 26 · B: CS 4, RST 27, DIO0 25 · 3.3 V only · antennas on before TX

Quantum plans where rescuers put relays (Scenario A)

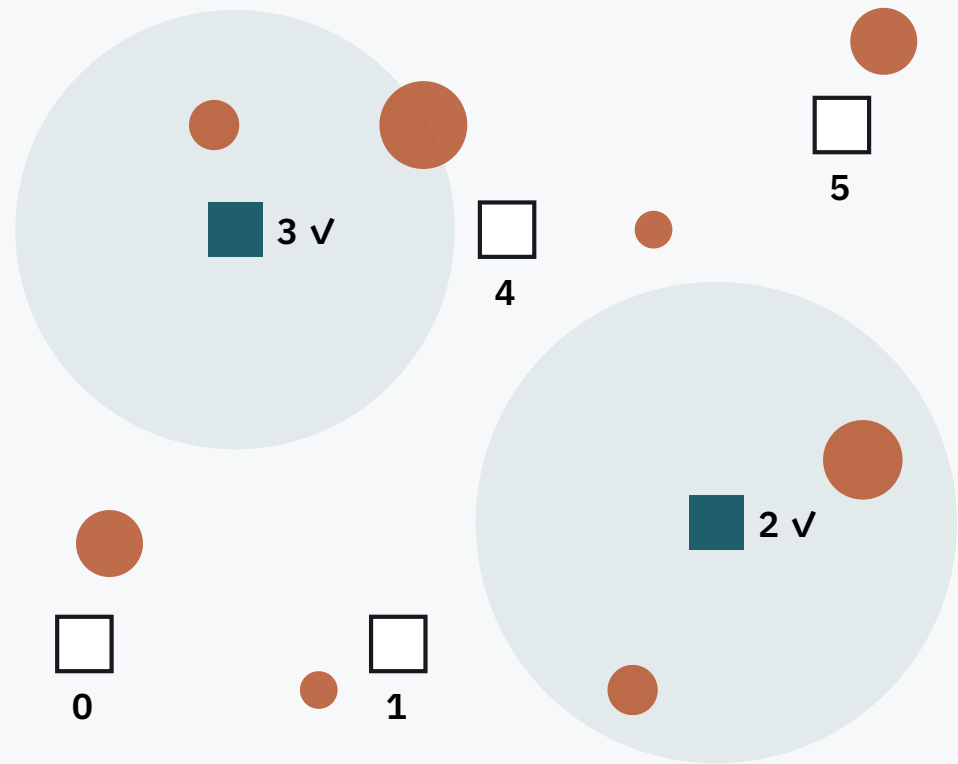
The problem: a team carries only a few relays. Put them in the wrong place and survivors stay cut off. Choosing K sites out of N is an optimisation that **grows explosively** with map size.

Our approach: turn the phone heatmap into survivor clusters → write it as a **QUBO** → solve with **QAOA on Qiskit** → show the chosen sites on the dashboard.

OUR RUN (QISKIT AER)	RESULT
6 sites, place 2 relays	6 qubits, p = 2
QAOA's choice	sites 2 + 3 → 13 of 21 people
Exact answer	same ✓

At this size a laptop is just as fast; we show the method works, not a speed-up. The radio network itself is fully classical.

Relay planner · 6 sites · 8 survivor clusters



A lifeline when everything else is down

0

towers, internet, cloud or apps needed

Any phone

or a shared tablet at the SOS point

2 lights

sent → read by SOS

WHO BENEFITS

Trapped survivors	reach rescuers from where they are
Displaced people	ask for help or tell family they're safe
Rescue teams	heatmap + urgent list instead of searching blind
Authorities	live picture of needs across the area

ROADMAP

Now	1 ESP32 + 2 LoRa: SOS page, link, receipt, dashboard
Next	3+ boards: real relay hops, phone-signal heatmap
Then	load + failure tests, measure delay, QAOA planner in dashboard
Pilot	district drill with an SDMA partner (proposed)

"When the towers fall, every phone can still call for help."

Team VOID
THANK YOU